



Family account <galguerrero@gmail.com>

Simulator Architecture Roadmap (Part 2)

1 mensaje

Family account <galguerrero@gmail.com>

22 de agosto de 2026 a las 22:18

Para: Family account <galguerrero@gmail.com>

Retr01 Simulator: Architecture & Optimization Roadmap (Part 2)

This document builds upon your current implementation (`w65c02s.c`, `as6c62256.c`, and `island_abc.c`). You have successfully separated the physical electrical wiring from the internal chip logic.

Here are the critical architectural questions you need to answer as you expand this from "Island ABC" to the full motherboard, followed by concrete optimization strategies for your exact C code.

Phase 1: Architectural Questions for the Full Board

1. Combinatorial Logic (The Zero-Delay Problem)

Your current loop in `island_abc.c` evaluates the CPU, wires the memory, ticks the CPU, and re-wires. This works because the CPU and SRAM are relatively self-contained.

- **The Question:** What happens when you introduce the ATF22V10 PLD?
- **The Context:** PLDs and 74HC chips do not have a clock (`tick`); they are purely combinatorial (`eval`). If the CPU changes A15, the PLD must immediately pull `/CE` low, which must immediately tell the SRAM to output data to the D bus, which the CPU must read on the next clock edge.
- **The Challenge:** How many times will you run `wire_memory()` and `eval()` per clock half-step to ensure the electrical signals "settle" across chained logic gates before the clock ticks?

2. High-Z and Bus Conflicts (The Pull-up Resistor)

You correctly implemented `r01s_bus_hiz(e, "DQ", 8)` in the SRAM when `/CE` is high.

- **The Question:** If both the CPU and the SRAM are in a High-Z state, what does `r01s_entity_sense` return?
- **The Context:** Real motherboards use physical pull-up resistors on the data bus. If no chip is driving the bus, the voltage floats to HIGH (`$FF`). Does your `r01s_bus_read` implementation default to `0xFF` if all connected pins are `R01S_LVL_Z`, or does it return `0x00`?

3. Clock Skew and Domain Crossing

You have a `phi2` clock for the 6502, but your ATmega1284P will be running on a separate 20 MHz crystal oscillator.

- **The Question:** How will your main simulator loop handle two separate, asynchronous clocks?
- **The Context:** Will you use a lowest-common-multiple virtual master clock (e.g., 40 MHz) and use counter variables to step the 20 MHz and 8 MHz chips at their respective intervals?

Phase 2: The Optimization Playbook (Targeting Your Code)

Right now, your priority is proving the hardware works. **Do not optimize yet.** > "Once the system boots and you can see a pixel on the screen, you can profile the C code and start replacing the heavy pin-level string lookups with direct memory pointers or bitmasks for a massive speed boost."

When you finally reach that milestone and the frame rate is sitting at 2 FPS, here is exactly how you will optimize the code you just wrote.

Target 1: The `snprintf` Trap

The Current Reality: Look at `copy_bus_named()` in `island_abc.c`. For a 16-pin address bus, it executes `snprintf` 32 times, followed by string comparisons inside `r01s_entity_drive` and `r01s_entity_sense`. It does this *thousands of times per second*. **The Optimization:** Instead of looking up pins by string during the simulation loop, you look them up *once* during a `board_init()` phase and cache their integer indices.

```
// Slow (Current)
r01s_entity_drive(dst, "A15", r01s_entity_sense(src, "A15"));

// Fast (Future Optimization)
// Setup phase: int pin_cpu_a15 = get_pin_id(cpu, "A15");
dst->pins[pin_ram_a15].level = src->pins[pin_cpu_a15].level;
```

By bypassing the string names and directly copying the `level` variable from one array index to another, you turn thousands of CPU cycles into a single memory assignment.

Target 2: Bypassing the Pins Entirely

The Current Reality: The CPU breaks a 16-bit integer into 16 boolean pin states. `island_abc.c` copies those 16 booleans to the SRAM. `as6c62256.c` reads those 16 booleans and reassembles them back into a 16-bit integer via `r01s_bus_read(e, "A", 15)`. **The Optimization:** Once you know your board routing works flawlessly, you stop simulating the copper traces. You create a shared motherboard struct that contains a single `uint16_t address_bus`. The CPU writes directly to it: `mb->address_bus = c->ab;`. The SRAM reads directly from it: `addr = mb->address_bus;`. This entirely eliminates the 16-pin iteration loops inside `cpu_drive_bus` and `sram_eval`.

Target 3: Inlining the Glue

The Current Reality: You are simulating the physical logic gates step-by-step. **The Optimization:** Once Island ABC is proven, you can replace the `wire_memory` function with pure C bitwise logic.

```
// Instead of propagating pins through a simulated PLD chip:
if (mb->address_bus >= 0x0000 && mb->address_bus <= 0x7FFF) {
    sram_chip->ce = R01S_LVL_L; // Direct memory mapped chip select
} else {
    sram_chip->ce = R01S_LVL_H;
}
```

The Milestone Mindset

Your current pin-level code is the absolute best way to catch hardware architecture bugs *before* you order a real PCB. Accept the slow frame rate. Use this literal simulation to verify that your CPU phase, multiplexers, and SRAM /CE logic align perfectly. Once you see a simulated pixel, you pull the trigger on these three targets and watch the frame rate explode into the hundreds.